

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Шаповалов Сергей

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service

Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом

Спроектируйте разделение вашей БД на обособленные БД

Спроектируйте и опишите endpoints, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI

Опишите варианты запросов и ответов, задокументируйте возможные ошибки

В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

1) Список микросервисов

- Auth Service

- Отвечает за аутентификацию, выпуск JWT, верификацию токенов.

- БД: auth_db (хранение сессий/refresh-токенов/blacklist при необходимости).

- User Service

- Управление профилями пользователей, роли.

- БД: users_db (таблица users, roles).

- Restaurant Service

- Управление ресторанами, меню, фотографии, типы кухни.

- БД: restaurants_db (restaurants, menu_items, cuisine_types, photos).

- Reservation Service

- Управление бронированиями (создание, статус, история).
- БД: reservations_db (reservations).

- Review Service

- Управление отзывами.
- БД: reviews_db (reviews).

- API Gateway (опционально)

- Единственная точка входа для клиентов; проксирует запросы к сервисам, проверяет JWT, агрегация.

2) Database-per-service — разделение БД

Каждый сервис имеет свою собственную базу данных (рекомендуется отдельный экземпляр/схема):

- users_db: хранит только данные пользователей и роли
- restaurants_db: данные ресторанов, меню, фото
- reservations_db: бронирования с ссылкой по id пользователя и id ресторана (только id, без денормализации данных)
- reviews_db: отзывы с userId и restaurantId
- auth_db: опционально, хранение refresh/blacklist

3) Взаимодействие между сервисами

- Синхронные REST-запросы для чтения справочных данных (например, ReservationService может запросить RestaurantService /restaurants/{id} для проверки существования ресторана).
- Аутентификация: клиент получает JWT от Auth Service; шлюз и сервисы валидируют токен (локально через секрет или через Auth Service `/auth/verify`).

- Асинхронные события через Message Broker (RabbitMQ/Kafka) для рассылки изменений (например, когда ресторан удалён — RestaurantService публикует событие `restaurant.deleted`; ReviewService/ReservationService слушают и очищают связанные данные).

5) Эндпоинты для межсервисного взаимодействия (с минимумом)

- Auth Service

- POST /auth/verify

- Тело: { token: string }

- Ответ 200: { userId: number, exp: number }

- Ошибки: 401 Unauthorized (invalid/expired)

- User Service

- GET /internal/users/{id}

- Ответ: { id, firstName, lastName, email }

- Ошибки: 404

- Restaurant Service

- GET /internal/restaurants/{id}

- Ответ: { id, name, address, priceRange }

- Ошибки: 404

- Reservation Service

- POST /internal/reservations/validate

- Проверить возможность бронирования (опционально)

- Ответ: 200 { ok: true } или 409 { ok: false, reason }